Post-launch connections: analytics, search, behaviour tracking and the deploy that carries them

How a freshly launched site gets its GA4, Search Console and Hotjar wired up and verified, almost all of it through Claude prompts. If you remember one thing: **an env change is not live until the next build, and nothing is verified until you have seen it in the live HTML**.

SOP ID	HGM-SOP-WEB-003	EFFECTIVE	2026-09-11
VERSION	1.0	NEXT REVIEW	2027-03-11
OWNER	Web & Content Specialist (Kyle Zinger)	DURATION	Spread across launch week
TRIGGER	Once per site, in launch week		

01 PURPOSE AND SCOPE

Purpose

Gets a newly launched site reporting to Google Analytics 4, verified in Google Search Console and recording in Hotjar, with each change published and checked on the live site.

In scope

Netlify-hosted Next.js sites built from the HGM template, once the domain is live. HGM-SOP-WEB-005 hands the site over here.

Out of scope

The launch itself (DNS cutover, go-live), the client-facing analytics walkthrough in `docs/ANALYTICS-SETUP.md`, and paid media tags beyond the Meta Pixel block already in the codebase.

ALWAYS TRUE

Five standing rules

True on every post-launch session.

1

Start from the live main, on your own branch

Task 1.1 proves main is what's live and makes your branch. Nothing is changed on main directly.

2

An env change is not live until the next build

Redeploy after every change to `GA_MEASUREMENT_ID`, `HOTJAR_SITE_ID` or `GOOGLE_SITE_VERIFICATION`, then check the live HTML.

3

Analytics variables go in Netlify's Production context only

Never in `.env.local`, so dev and preview traffic stays out of the client's reports.

4

One web team login for everything Google

Create Analytics and Search Console as `hgmwebteam@gmail.com`, so one login holds both and task 4.6 works first time.

5

Never print or paste an env value

Check lengths, not contents. A value pasted into a chat, an Asana comment or a prompt is a key that has to be rotated.

02 PREREQUISITES

1 **VS Code or the Claude desktop app, with Claude Code signed in**

Every task runs from a Claude prompt. First time on this computer? HGM-SOP-WEB-002 Phase 1 installs the tools.

2 **The Netlify connector**

Connected in Claude, with access to the client's Netlify team. Claude uses it to check what's live, set environment variables and redeploy.

3 **The site's .env.local entry in 1Password**

Task 1.2 copies it in (HGM-SOP-WEB-001).

4 The web team login

`hgmwebteam@gmail.com`, for Google Analytics and Search Console, and invited to the client's Netlify, Supabase and Resend accounts (task 1.3).

5 The agency Hotjar login

`hgmwebteam`. It holds one organisation per client (Hotjar's container for that client; plan limits and seats are per organisation); sites are added inside the client's organisation, never at the top level.

03 PROCEDURE

What are you trying to do?

Pick the phase that matches your situation and jump straight to its tasks.

PHASE 1

Get the live site and your branch

Tasks 1.1–1.3 · start of every post-launch session

PHASE 2

Check the analytics code

Tasks 2.1–2.2 · one prompt, once per site

PHASE 3

Google Analytics 4

Tasks 3.1–3.8 · find or create, tag, settings, verify

PHASE 4

Google Search Console

Tasks 4.1–4.6 · verify, sitemap, indexing, owner, GA link

PHASE 5

Hotjar

Tasks 5.1–5.6 · check, install, privacy, invite

PHASE 6

Structured data check

Task 6.1 · one prompt checks and fixes

Phase 1 — Get the live site and your branch

Start of every post-launch session. One prompt gets the latest code, proves it is what's live and makes your branch.

1.1 Open the site and make your branch

Open the site's folder in VS Code (an empty folder if this computer has never had the site), or start Claude Code in the Claude desktop app, then paste the prompt. Your branch is your own copy of the code to work on, named `post-launch`.

CLAUDE PROMPT — FILL IN THE THREE ANGLE-BRACKET VALUES

```
I'm starting post-launch connections (HGM-SOP-WEB-003) for
<client>, live at https://<domain>. Get me set up:

1. If this folder isn't the site's repository, clone
   https://github.com/hgmwebteam/<site-name>.git and open it.
2. Switch to main and pull the latest. If there are changes
   here I don't recognise, stop and show me. Discard nothing.
3. Prove main is what's live: check <domain> points at this
   site's Netlify project, and that the latest production
   deploy was built from origin/main's commit. If the Netlify
   connector can't see the site, stop and tell me to do
   task 1.3 first.
4. Create a branch named post-launch from main and switch
   to it.

Tell me plainly if anything doesn't match. Make no other
changes.
```

1.2 Add the .env.local file, then have Claude check it

Copy the site's `.env.local` entry from 1Password into a file named `.env.local` in the project folder (HGM-SOP-WEB-001), then paste the prompt. A file copied from the template looks like a broken site: empty listings, missing photos. If keys are blank, the 1Password entry needs updating (WEB-001 Phase 2).

CLAUDE PROMPT — LENGTHS ONLY, NEVER VALUES

```
Check .env.local: list every key with the LENGTH of its value.
Never print a value. Tell me which keys are blank. Do not
fetch or write any secret yourself.
```

1.3 Check the web team is on the client's Netlify, Supabase and Resend

Sign in to each as `hgmwebteam@gmail.com` and confirm the client's Netlify team, Supabase project and Resend account are listed. If one is missing, invite `hgmwebteam@gmail.com` to it the way HGM-SOP-WEB-004 tasks 2.1, 4.2 and 4.7 do. The team's automations connect through this account, with nothing else to set up.

Phase 2 — Check the analytics code is in place

Once per site. One prompt reads the code; you act only if it changed something.

2.1 Ask Claude to check the analytics code

The site's page head carries three blocks, each switched on by a Netlify variable and silent while it is unset. The prompt confirms they are there, adds any that is missing, and confirms there is no GA4 purchase event: bookings are reported to Meta from the server, so a browser purchase event would count every booking twice.

CLAUDE PROMPT — PASTE AS IT IS

```
Check this site's analytics code and tell me what you find
before you change anything. Change code only on this branch.

1. app/layout.tsx must have three blocks, modelled on the
   Meta Pixel block. Each reads a server-side variable (no
   NEXT_PUBLIC_ prefix), checks the value's shape, and
   renders nothing while the variable is unset:
   - GA_MEASUREMENT_ID (G-...): gtag.js plus gtag('config'),
     strategy lazyOnload
   - HOTJAR_SITE_ID (digits): the Hotjar snippet (hjsv 6),
     strategy lazyOnload
   - GOOGLE_SITE_VERIFICATION (a token): the
     google-site-verification meta tag, through Next's
     metadata.verification.google
2. All three names must be in .env.example and in the docs'
   environment list.
3. There must be NO GA4 purchase event. Bookings reach Meta
   from the server, so one would double count.

If anything is missing, add it on the Meta Pixel pattern,
commit it on this branch and tell me what you changed. If
everything is there, say so and change nothing.
```

2.2 If Claude changed code, take it live before Phase 3

If Claude changed nothing in 2.1, go to Phase 3. Otherwise the variables in Phases 3 to 5 do nothing until this code is live, so paste the prompt. It opens a pull request (a request to merge your branch into main), which you merge on GitHub the way HGM-SOP-WEB-002 Phase 6 does, then checks the live site was rebuilt from it. Task 6.1 comes back here if it changes code too.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Take my branch live:

1. Push this branch and give me the link to open a pull
   request into main. Do not merge it yourself.
2. When I tell you it's merged, switch to main, pull, and use
   the Netlify connector to confirm the latest production
   deploy was built from the merge commit.
3. If production did not rebuild, deploy main with the
   Netlify connector from a clean copy of main's committed
   files only. Never upload this folder: it holds .env.local
   and build files, and Netlify returns errors.
4. Fetch https://<domain>/ and confirm the change is live.

Then put me on a new post-launch branch made from main.
```

Phase 3 — Google Analytics 4

Signed in as `hgmwebteam@gmail.com` throughout. Use the client's own account if they have one; create one for them if they don't.

3.1 Find the client's Google Analytics, or create an account for them

Ask the account manager whether the client already has Google Analytics, and paste the prompt to look for it in the code. **If they have an account**, send the message below, then use their property for this domain (or create one inside their account) and go to 3.3. **If not**, open **Admin → Create → Account**, name it after the client, then add a property named after the site with a web data stream for the apex domain (the address with no `www`).

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Check whether <client> already uses Google Analytics. Search
Resources/Page HTML (the old site's pages from WEB-004, if
this folder has them) and the live https://<domain>/ for a
GA4 ID (G-...), an old Universal Analytics ID (UA-...) or a
Tag Manager container (GTM-...). Tell me each ID you find and
where. Make no changes.
```

MESSAGE FOR THE ACCOUNT MANAGER

<client> already has Google Analytics. Please ask them to add hgmwebteam@gmail.com as an Administrator under Admin → Account access management, so we can connect their new site.

⚠ Creating a property in the client's account

Read the account name at the top of **Admin** first. A property lands in whichever account is showing, and the web team login sees every client's. Move a misplaced property; recreating it changes its measurement ID.

3.2 Add the client as Administrator

Only if you created the account in 3.1. **Admin → Account access management → Add users**, enter the client's Google login from the account manager, role **Administrator**.

3.3 Set GA_MEASUREMENT_ID and redeploy

Copy the measurement ID (the `G-…` code) from **Admin → Data streams** → your stream, then paste the prompt with it.

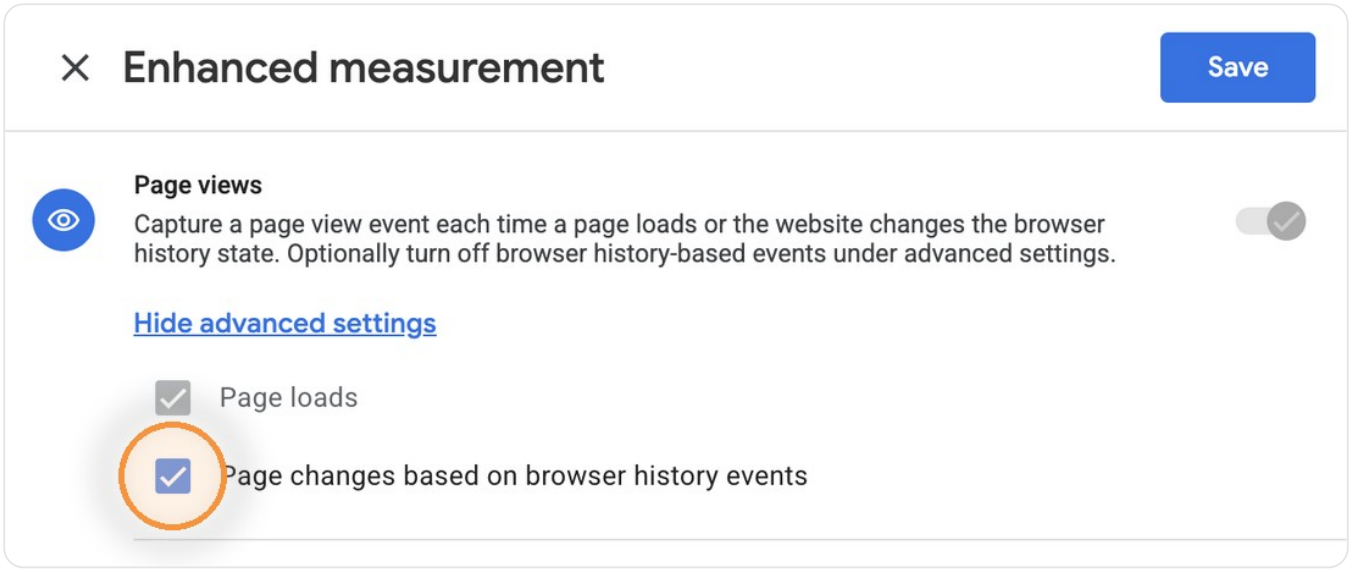
CLAUDE PROMPT — FILL IN THE G- ID AND THE DOMAIN

```
Using the Netlify connector, set GA_MEASUREMENT_ID to <G-ID>
in this site's Production context only (never .env.local),
then redeploy production from main. Never deploy by uploading
this folder: it holds .env.local and build files, and Netlify
returns errors.

When the deploy is live, fetch https://<domain>/ and confirm
<G-ID> is in both the gtag/js script URL and the
gtag('config') call. Retry every 30 seconds for up to
3 minutes before you call it a failure.
```

3.4 Turn on page views for browser history events

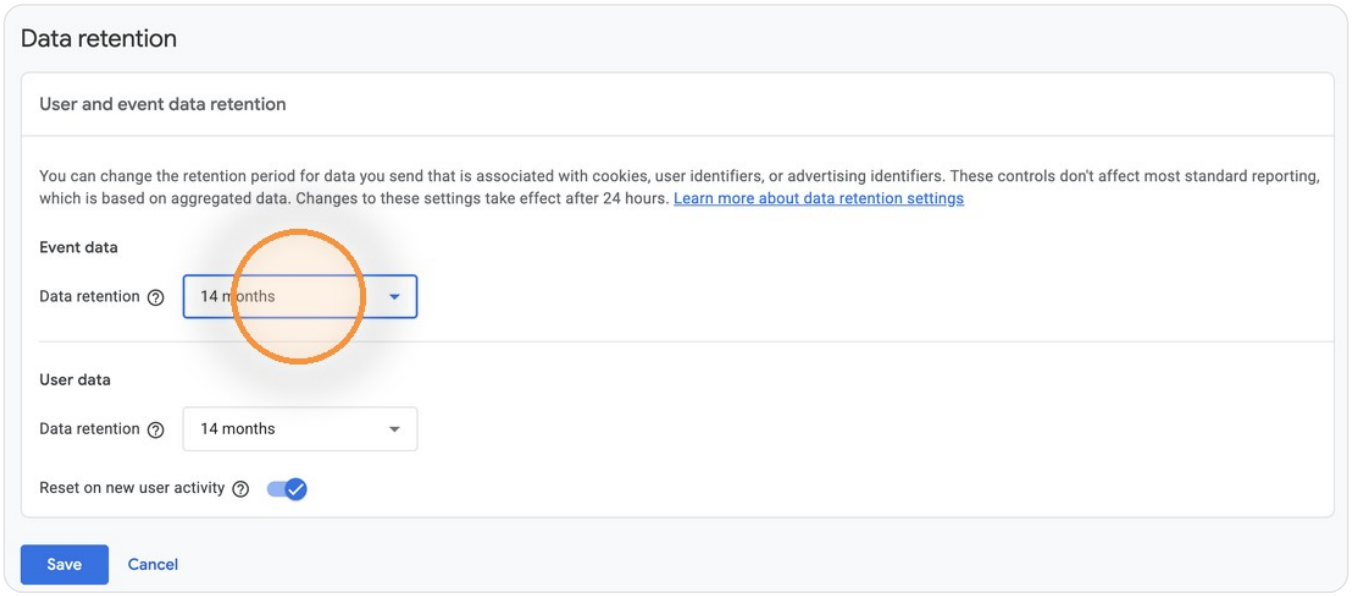
Admin → Data streams → your stream → **Enhanced measurement** → gear icon → **Page views → Show advanced settings → Page changes based on browser history events: ON**. The site fires one page view per full load and relies on this setting for client-side navigations.



Enhanced measurement → Page views → Show advanced settings: tick Page changes based on browser history events, then Save.

3.5 Set data retention to 14 months

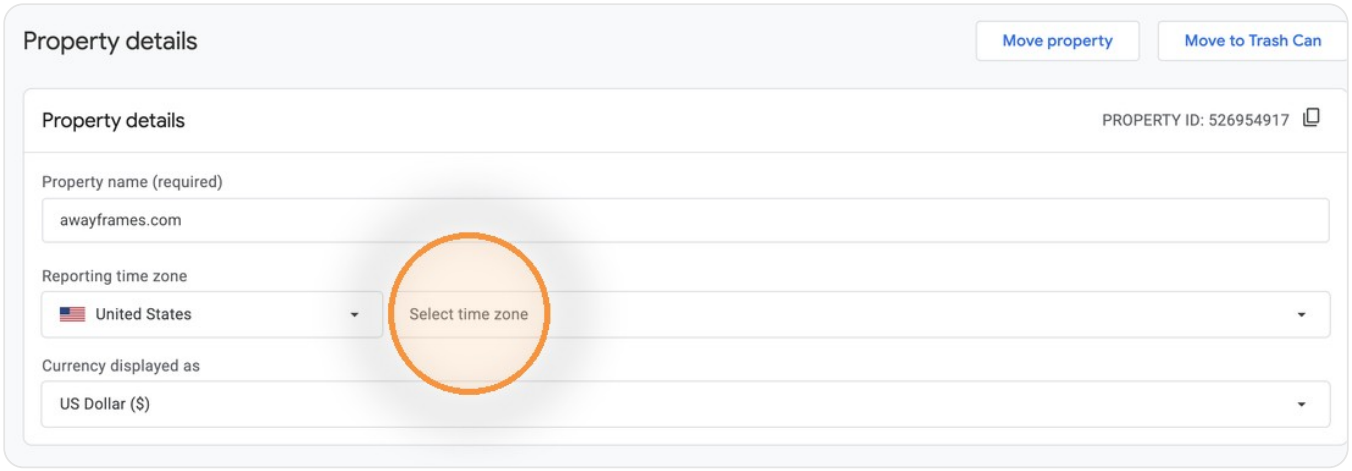
Admin → Data collection and modification → Data retention → Event data retention: 14 months. The default is 2.



Admin → Data retention: Event data set to 14 months, then Save.

3.6 Set the property time zone and currency

Admin → Property details. Time zone = the site's, from `BRAND.timezone` in the codebase. Currency **USD**.



Admin → Property details: choose the site's time zone (still unset here) and US Dollar.

3.7 Define internal traffic and set the filter to Active

Admin → Data streams → your stream **→ Configure tag settings → Define internal traffic**: add the office and home IPs. Then **Admin → Data filters → Internal Traffic** and set its state to **Active**. It ships in Testing, which does nothing to reports.

3.8 Verify in Realtime with a real visit

Open the live site in a browser, then **Reports → Realtime**. You should appear as an active user within a minute.

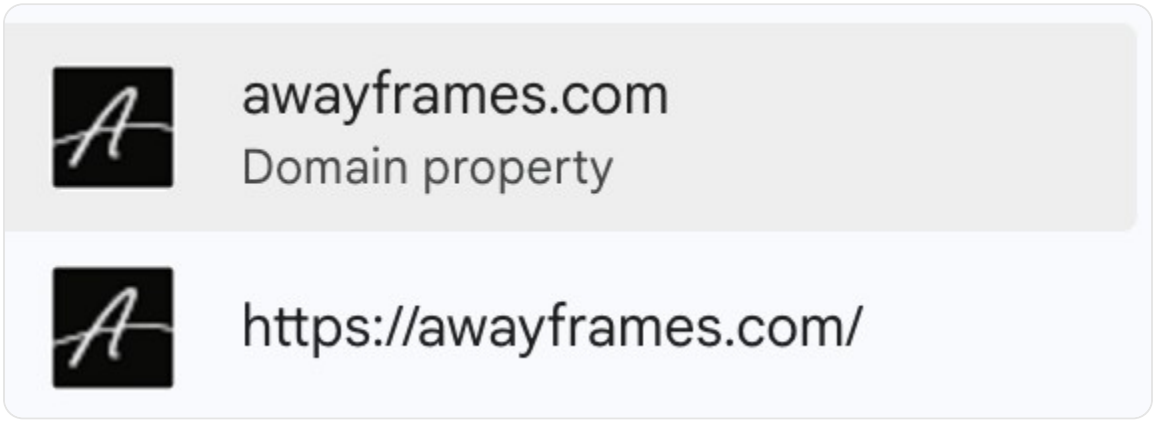
> ⓘ **Two things that look like failure and are not**
>
> The "Data collection isn't active" banner can take 48 hours to clear: ignore it if Realtime works. Google's **Test your website** button may not see the tag, because it loads after `window.load`.

Phase 4 — Google Search Console

Signed in as `hgmwebteam@gmail.com`, the same login as Phase 3. A URL-prefix property (one exact address) verified with an HTML tag.

4.1 Add a URL-prefix property and copy the HTML tag token

Add property → URL prefix, enter `https://<domain>` (the apex; www and http redirect to it). If the Domain dialog opens, click **try a URL prefix property instead**. Under the verification methods choose **HTML tag** and copy the `content` value only, not the whole tag.



The property list: a URL-prefix property shows as its full address (`https://…`); a Domain property says so underneath.

4.2 Set GOOGLE_SITE_VERIFICATION, redeploy, then click Verify

Paste the prompt with the token. When Claude confirms the tag is live, go back to Search Console and click **Verify**.

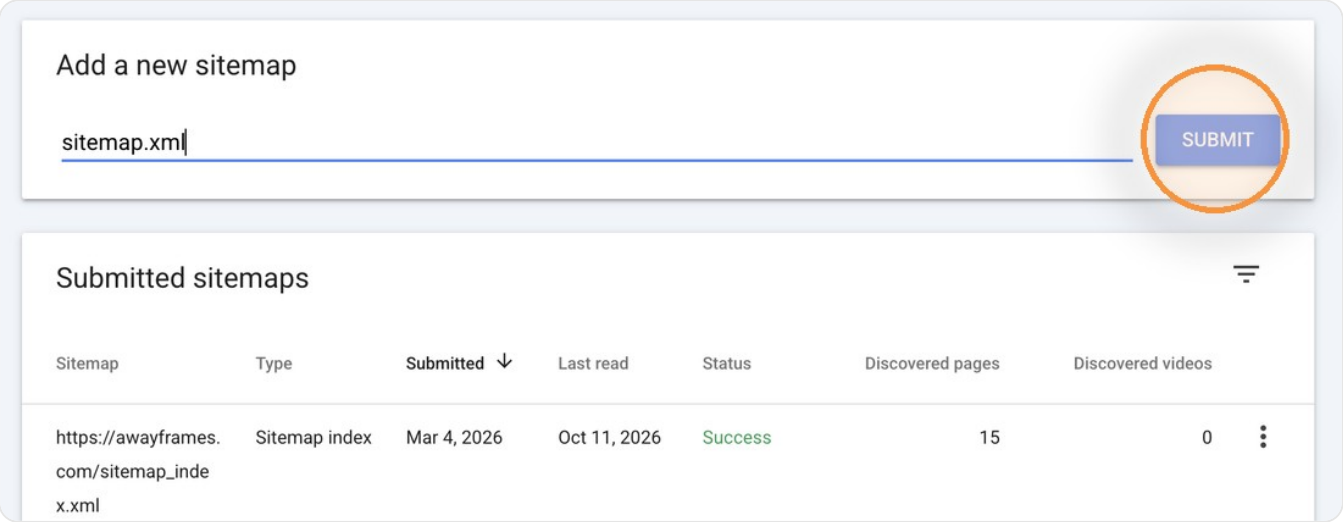
CLAUDE PROMPT — FILL IN THE TOKEN AND THE DOMAIN

```
Using the Netlify connector, set GOOGLE_SITE_VERIFICATION to
<TOKEN> in this site's Production context only (never
.env.local), then redeploy production from main. Never
deploy by uploading this folder: it holds .env.local and
build files, and Netlify returns errors.

When the deploy is live, fetch https://<domain>/ and confirm
the google-site-verification meta tag's content equals
<TOKEN>. Retry every 30 seconds for up to 3 minutes before
you call it a failure.
```

4.3 Submit the sitemap

Sitemaps → Add a new sitemap: `sitemap.xml`, then **Submit**. Pages set to noindex are left out of it on purpose.



Sitemaps → Add a new sitemap: type sitemap.xml and Submit. The submitted sitemap should read Success.

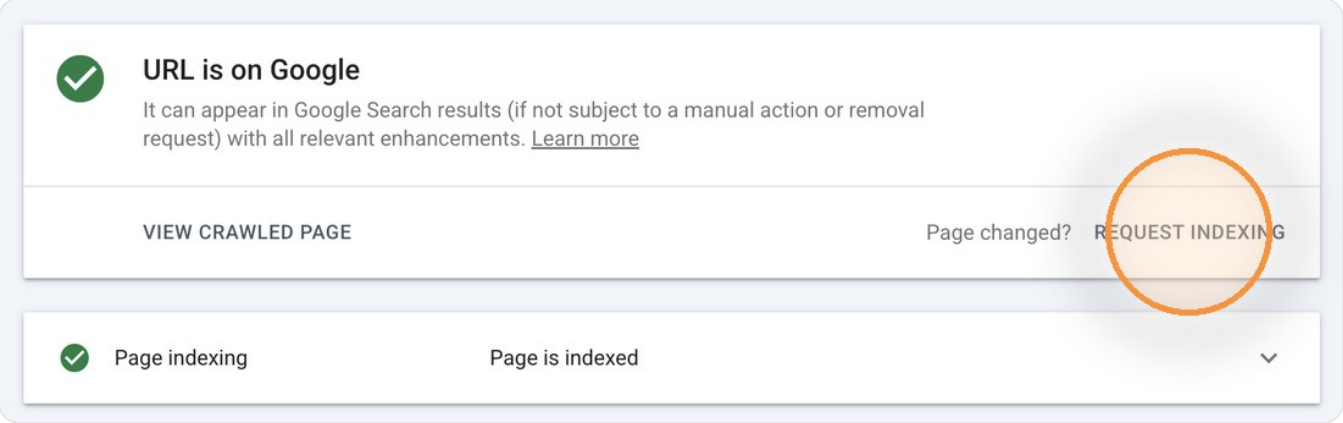
4.4 Request indexing for the main pages

Request indexing for at least the pages below, and every other page too if you want. Paste the prompt for their full addresses, then for each one: **URL Inspection** → paste it → **Request indexing**. The daily quota is small: if Search Console says you have reached it, request the rest the next day.

- The home page: `https://<domain>/`
- The stays page: `https://<domain>/stays`
- Every listing page, for example `https://<domain>/listing1`, `https://<domain>/listing2` and so on

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

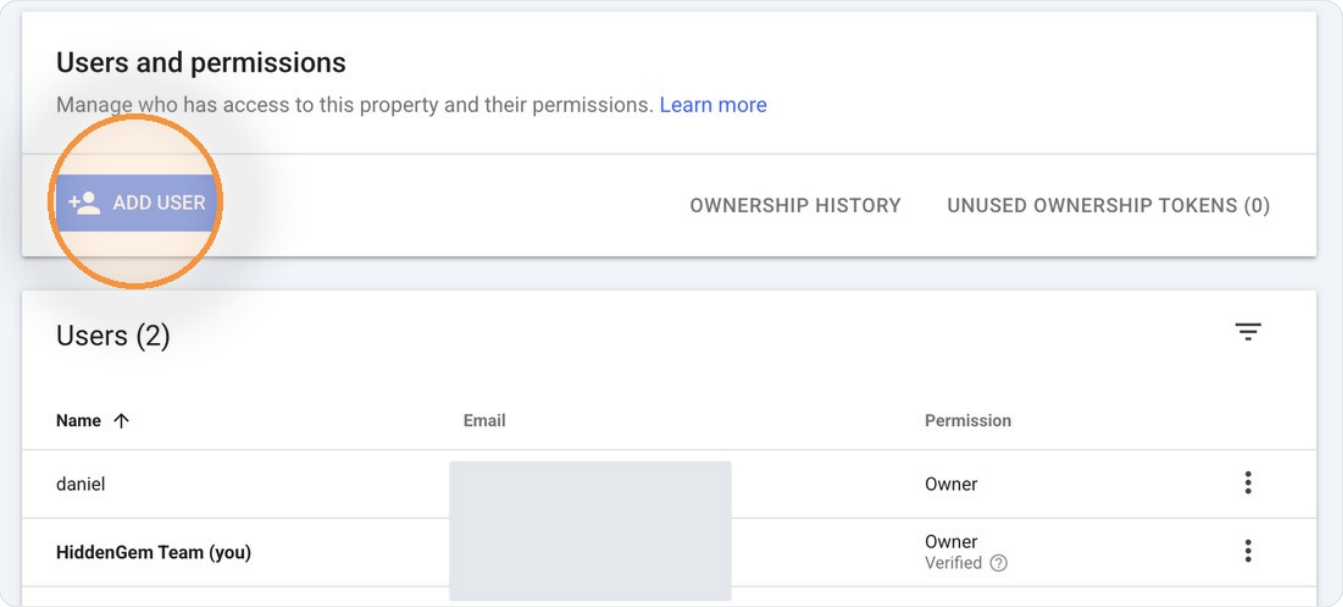
```
Read https://<domain>/sitemap.xml and list, one per line, the
full address of the home page, the stays page and every
listing page, then every other page. Make no changes.
```



URL inspection's result card: Request indexing is on the right.

4.5 Add the client as Owner

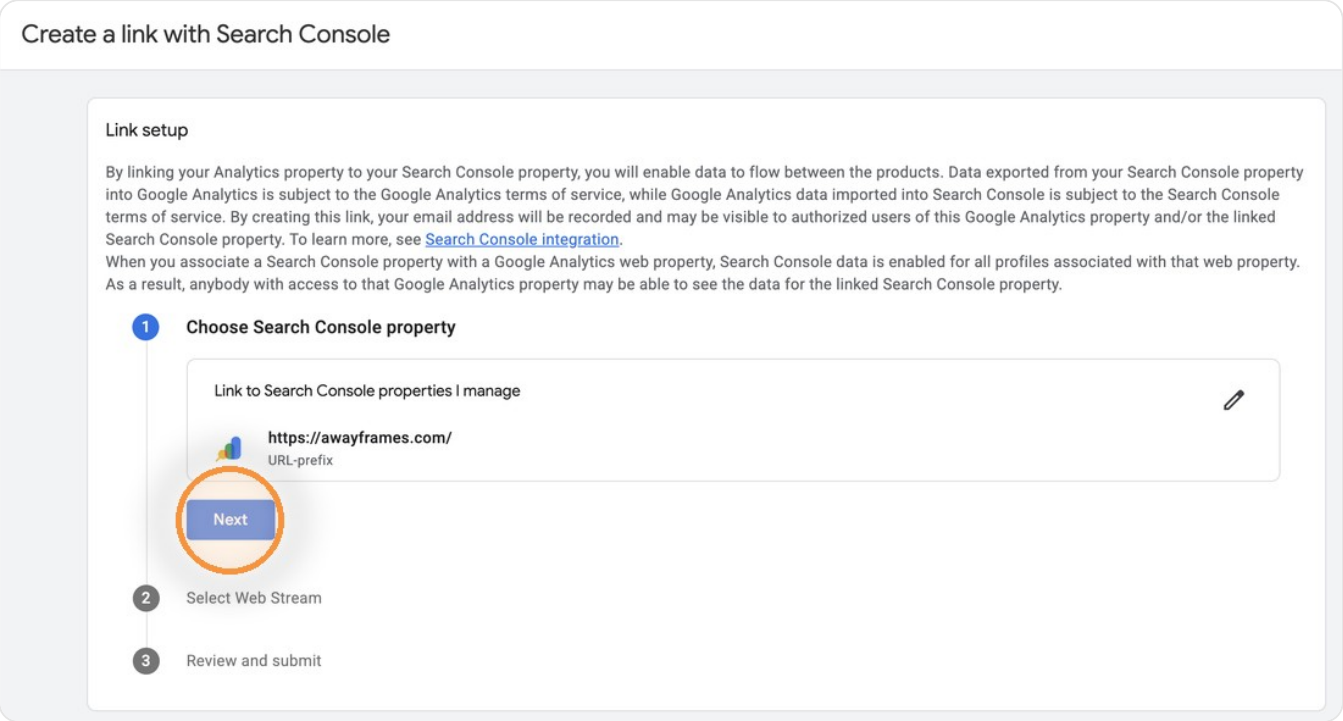
Settings → Users and permissions → Add user, enter the client's Google login, permission **Owner**.



Settings → Users and permissions: Add user, then choose Owner. Emails hidden here.

4.6 Link Google Analytics to Search Console

In Google Analytics: **Admin → Product links → Search Console links → Link**, choose the property you verified in 4.2, then **Next**. The link needs a login that owns the Search Console property and is Editor or above in Analytics; `hgmwebteam@gmail.com` is both.



GA Admin → Product links → Search Console links → Link: choose the verified property, then Next.

Phase 5 — Hotjar

Check first, then the client's organisation, install, privacy settings before the first recording, and a real browser visit to verify.

5.1 Check whether the client already runs Hotjar

So you never create a second Hotjar site for one that already has one. Get the client's other websites from the account manager, then paste the prompt.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

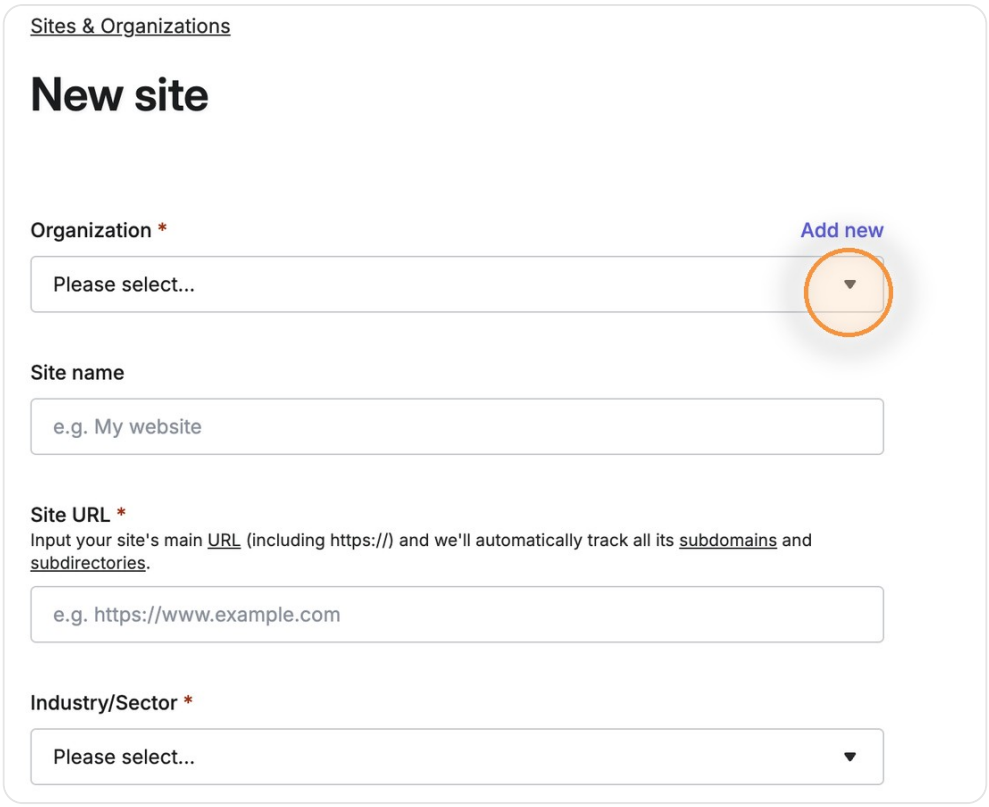
```
Check whether <client> already uses Hotjar. Fetch the home
page of each of these sites and look for an hjid:
<domain-1>, <domain-2>, <domain-3>.
Also search Resources/Page HTML (the old site's pages, if
this folder has them) and any Google Tag Manager container
you can see. Report each hjid you find and where. Make no
changes.
```

5.2 Open the client's organisation in Hotjar

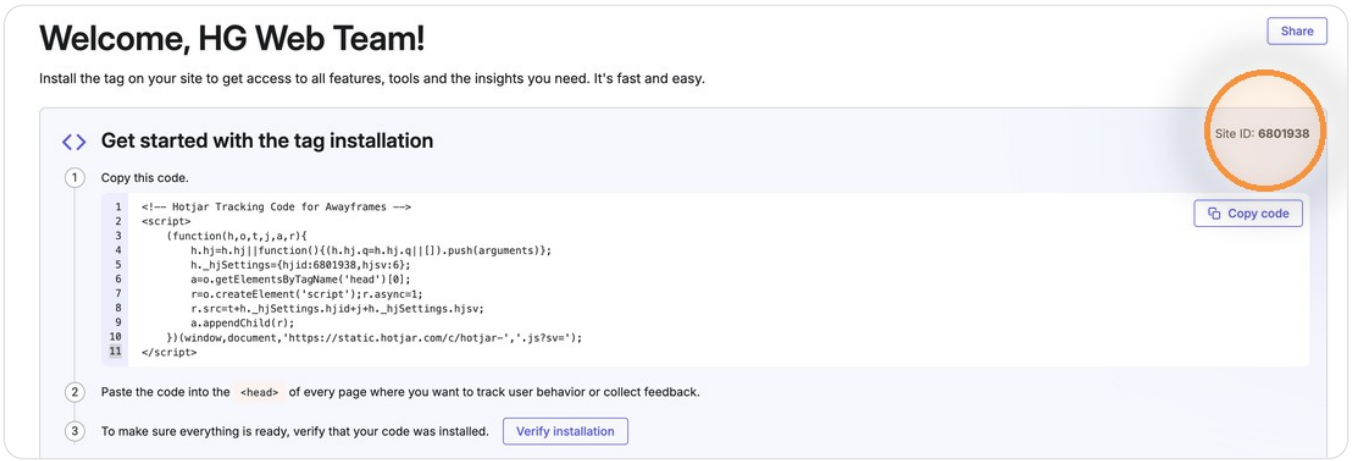
Log in as `hgmwebteam`. Switch to the client's organisation. If none exists, choose **Add as a New Client Organization** and name it after the **client**, not the site: plan limits and team seats are per organisation.

5.3 Add the site, then set HOTJAR_SITE_ID and redeploy

Inside the client's organisation, add the site for the apex domain and copy its numeric Site ID, then paste the prompt with it.



Hotjar → Sites & Organizations → New site: choose the client's organisation first, never the top level.



Once the site is added, its Site ID is at the top right of the install page. That number is HOTJAR_SITE_ID.

CLAUDE PROMPT — FILL IN THE SITE ID AND THE DOMAIN

```
Using the Netlify connector, set HOTJAR_SITE_ID to <SITE-ID>
in this site's Production context only (never .env.local),
then redeploy production from main. Never deploy by uploading
this folder: it holds .env.local and build files, and Netlify
returns errors.

When the deploy is live, fetch https://<domain>/ and confirm
the Hotjar snippet carries <SITE-ID>. Then fetch
https://static.hotjar.com/c/hotjar-<SITE-ID>.js?sv=6 and
confirm it returns 200 and names site_id <SITE-ID>.
```

5.4 Visit the site in a real browser

Open the live site in a browser with no ad blocker (Brave and uBlock hide Hotjar) and click through two pages. Hotjar marks a site installed only after its script runs in a real browser; Claude's fetch does not count.

5.5 Fix the privacy settings before the first recording

In the site's settings: turn **keystroke capture off** (it was on by default on Ridge & Falls). Add URL exclusions for paths containing `/admin`, `/manual`, `/checkout` and `/confirmation`. Add the office IPs to **IP blocking**.

The confirmation page prints guest name and email as text

Without the exclusion, a session recording captures personal data. Set the exclusions before any real traffic is recorded.

5.6 Invite the client

Team → Invite, and enter the client's login.

Phase 6 — Structured data check

Once, after Phases 3 to 5. One prompt checks the stays page and a listing page, and fixes the code if it needs it.

6.1 Ask Claude to check the structured data

Structured data is the hidden description of each page that search and answer engines read. Paste the prompt, then test the two addresses it gives you in Google's **Rich Results Test** (`search.google.com/test/rich-results`). If Claude changed code, take it live the way task 2.2 does, then test again.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Check the structured data (JSON-LD) on https://<domain>/stays
and one listing page.

1. Fetch both pages and pull out every JSON-LD block.
2. Look for one @id emitted twice with different content,
   such as a thin copy of each rental in the stays page's
   ItemList. Google merges the two and reports critical
   issues on every listing.
3. Check each entity against Google's documentation for its
   type, fetched now, not remembered: every required
   property, and the recommended ones where the data is real.

If you find a problem, fix it on this branch with one builder
function per entity (for example vacationRentalJsonLd in
lib/seo.ts), used by every page that mentions it. Leave out
anything you would have to invent, such as review dates or
bed types. Commit it and tell me what changed.

Finish by giving me both addresses to test in Google's Rich
Results Test. The aim is zero critical issues on each.
```

04 EXCEPTIONS AND EDGE CASES

A client change request arrives during launch week

The pattern, in order. Every example shipped on Ridge & Falls followed it.

1. **Locate the code from the screenshot or chat request**

   If the screenshot shows something "missing" on a dev box, check the data feed before the code: see task 1.2.

2. **Edit, then run the checks**

   Ask Claude to run `tsc`, `vitest` and `verify:brand`.

3. **Render through the real stylesheet for a one-look check**

   Use the Playwright harness so you are looking at the actual output, not a guess.

4. **Record the decision**

   In `CLAUDE.md` and the team manual's Updates.

5. **Commit, take it live, check it**

   Task 2.2 takes the branch live and confirms the change is on the live site.

A changed asset is a new filename

Static assets are cached immutable for a year. Ship `wordmark-light-compact.svg`, `collection-*.webp`, `local-favorites-mclemore-resort-v3.webp` — never overwrite in place.

Keep the client's swatches even when contrast fails

Document the contrast ratio at the declaration and tell the client the passing alternative. Do not silently change their colour.

Blur is often the sizes hint, not the file

The Local Favorites portrait crop was blurry because of the `srcset sizes` hint, not the images. Check the hint before regenerating assets.

If you are stuck

Ask Claude first, with the task number and what you see. Then Kyle Zinger (Web & Content Specialist) in the **Web Team** Google Chat group. Anything that needs the client — access to their Google Analytics, a missing invite — goes through the client's account manager, never the client directly.

05 FINAL CHECKLIST BEFORE HANDOFF

- [ ] Main confirmed as what's live; `post-launch` branch made from it; `.env.local` filled (lengths checked, not values)
- [ ] `hgmwebteam@gmail.com` is a member of the client's Netlify, Supabase and Resend accounts
- [ ] Analytics blocks present in `app/layout.tsx`, names documented, no purchase event; any code change merged and live
- [ ] GA4: the client's own account used, or a new one created for them with the client as Administrator
- [ ] `GA_MEASUREMENT_ID` set in Production → redeployed → tag in the live HTML → Realtime shows a visit
- [ ] History events on; retention 14 months; time zone and currency set; internal traffic filter Active
- [ ] Search Console URL-prefix property verified with the HTML tag; sitemap submitted
- [ ] Home page, stays page and every listing page requested for indexing

- [ ] Client added as Search Console Owner; Google Analytics linked to Search Console
- [ ] Hotjar site inside the client's organisation; `HOTJAR_SITE_ID` set → redeployed → installed after a real browser visit
- [ ] Hotjar keystrokes off, private pages excluded, office IPs blocked, client invited
- [ ] Structured data on the stays page and a listing page: zero critical issues in the Rich Results Test

06 RELATED SOPS

- HGM-SOP-WEB-001 — env files in 1Password
- HGM-SOP-WEB-002 — take a website change request from Asana to live (branch, preview, merge)
- HGM-SOP-WEB-004 — the build, where the web team is invited to the client's Netlify, Supabase and Resend
- HGM-SOP-WEB-005 — pre-launch and the cutover, which hands the site to this SOP

07 REVISION HISTORY

DATE	VERSION	AUTHOR	CHANGE SUMMARY
2026-09-11	0.1	Kyle Zinger	Working draft converted from the Ridge & Falls launch-week notes, for Brandon. Not yet approved.
2026-10-15	0.2	Kyle Zinger	Loom review layout (house style v1.9): roles, definitions and troubleshooting removed, an If you are stuck note added, and links to other SOPs open in the portal.
2026-10-15	0.3	Kyle Zinger	Ten screenshots from the Awayframes launch added to tasks 4.4–6.3 (Google Analytics, Search Console, Hotjar), cropped and ringed.
2026-10-19	1.0	Kyle Zinger	Rebuilt around Claude prompts. Code goes live through a merge, GA4 starts from the client's own account, and terminal commands, Google Signals and the ownership map are gone.

REMEMBER

"Prove you are looking at the live site before you change it. Then remember that nothing you set is live until the next build, and nothing is verified until you have seen it in the live HTML."

Questions?
Kyle — Web Team Google Chat group